

Gradient descent

- Optimization is a big part of machine learning. Almost every machine learning algorithm has an optimization algorithm at its core
- **Gradient Descent**
- Gradient descent is an optimization algorithm used to find the values of parameters (coefficients) of a function (f) that minimizes a cost function (cost).
- Gradient descent is best used when the parameters cannot be calculated analytically (e.g. using linear algebra) and must be searched for by an optimization algorithm.

Intuition of GD



- Think of a large bowl
- This bowl is a plot of the cost function (f).
- A random position on the surface of the bowl is the cost of the current values of the coefficients (cost).
- The bottom of the bowl is the cost of the best set of coefficients, the minimum of the function.
- The goal is to continue to try different values for the coefficients, evaluate their cost and select new coefficients that have a slightly better (lower) cost.
- Repeating this process enough times will lead to the bottom of the bowl and you will know the values of the coefficients that result in the minimum cost.

Gradient Descent Procedure

- The procedure starts off with initial values for the coefficient or coefficients for the function. These could be 0.0 or a small random value.
coefficient = 0.0
- The cost of the coefficients is evaluated by plugging them into the function and calculating the cost.
- **cost = f(coefficient)**
- or
- **cost = evaluate(f(coefficient))**
- The derivative of the cost is calculated. The derivative is a concept from calculus and refers to the slope of the function at a given point. We need to know the slope so that we know the direction (sign) to move the coefficient values in order to get a lower cost on the next iteration.
- **delta = derivative(cost)**

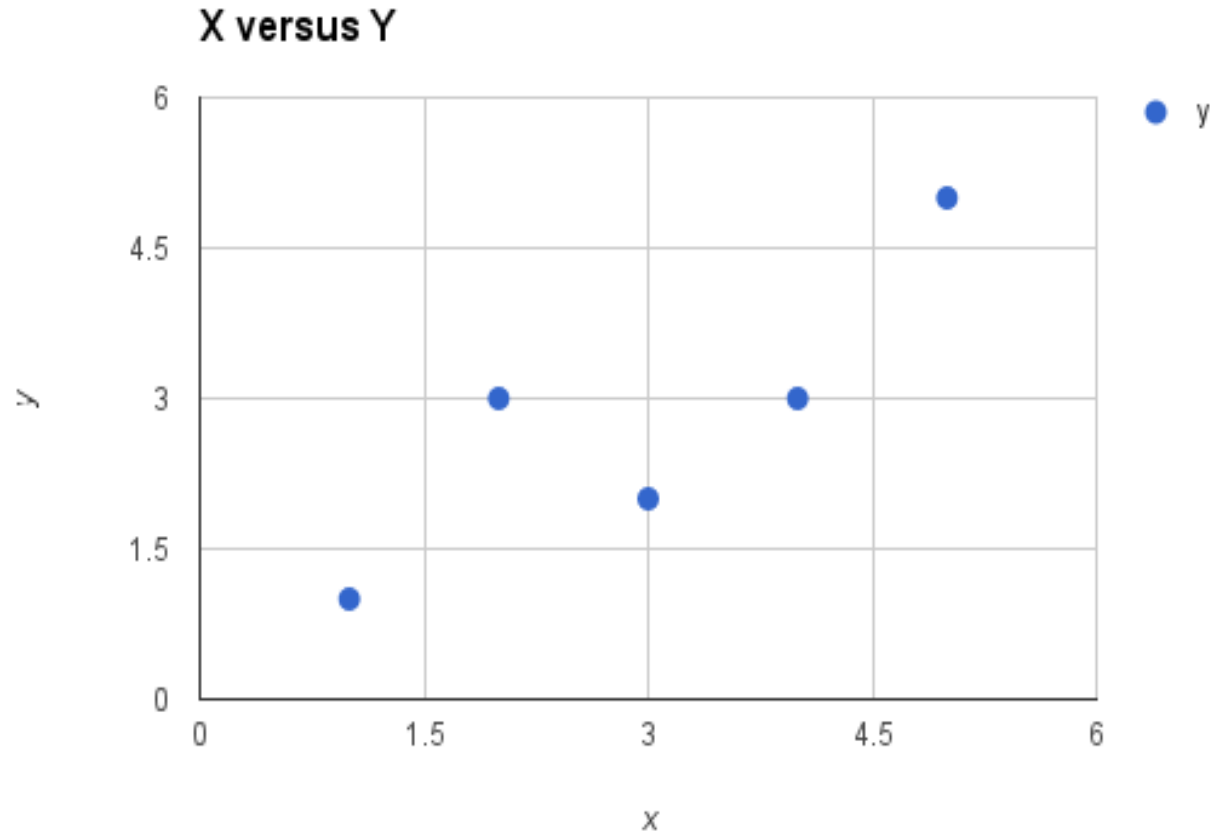
- we know from the derivative which direction is downhill, we can now update the coefficient values. A learning rate parameter (alpha) must be specified that controls how much the coefficients can change on each update.
- **$\text{coefficient} = \text{coefficient} - (\text{alpha} * \text{delta})$**
- This process is repeated until the cost of the coefficients (cost) is 0.0 or close enough to zero to be good enough.
- This is gradient descent approach.

- how to use Stochastic Gradient Descent to learn the coefficients for a simple linear regression model by minimizing the error on a training dataset.

simple scatter plot of x versus y.

Sample Data, x
is input, y is
target variable

1	x	y
2	1	1
3	2	3
4	4	3
5	3	2
6	5	5



We can see the relationship between x and y looks kind-of linear. As in, we could probably draw a line somewhere diagonally from the bottom left of the plot to the top right to generally describe the relationship between the data. This is a good indication that using linear regression might be appropriate for this little dataset.

- With simple linear regression we want to model our data as follows:
- $y = B0 + B1 * x$
- This is a line where y is the output variable we want to predict, x is the input variable we know and B0 and B1 are coefficients we need to estimate.
- B0 is called the intercept because it determines where the line intercepts the y axis. In machine learning we can call this the bias, because it is added to offset all predictions that we make. The B1 term is called the slope because it defines the slope of the line or how x translates into a y value before we add our bias.

Stochastic Gradient Descent

- Gradient Descent is the process of minimizing a function by following the gradients of the cost function.
- This involves knowing the form of the cost as well as the derivative so that from a given point you know the gradient and can move in that direction, e.g. downhill towards the minimum value.
- The way stochastic works is that each training instance is shown to the model one at a time. The model makes a prediction for a training instance, the error is calculated and the model is updated in order to reduce the error for the next prediction.

This procedure can be used to find the set of coefficients in a model that result in the smallest error for the model on the training data. Each iteration the coefficients, called weights (w) in machine learning language are updated using the equation:

$$w = w - \alpha * \text{delta}$$

Where w is the coefficient or weight being optimized, α is a learning rate that you must configure (e.g. 0.1) and gradient is the error for the model on the training data attributed to the weight.

Simple Linear Regression with Stochastic Gradient Descent

linear regression model is defined as follows:

$$y = B_0 + B_1 * x$$

Gradient Descent Iteration #1

Let's start with values of 0.0 for both coefficients.

$$B_0 = 0.0$$

$$B_1 = 0.0$$

$$y = 0.0 + 0.0 * x$$

We can calculate the error for a prediction as follows:

$$\text{error} = p(i) - y(i)$$

Where $p(i)$ is the prediction for the i 'th instance in our dataset and $y(i)$ is the i 'th output variable for the instance in the dataset.

We can now calculate the predicted value for y using our starting point coefficients for the first training instance:

$$x=1, y=1$$

$$p(i) = 0.0 + 0.0 * 1$$

$$p(i) = 0$$

Using the predicted output, we can calculate our error:

$$\text{error} = 0 - 1$$

$$\text{error} = -1$$

We can now use this error in our equation for gradient descent to update the weights. We will start with updating the intercept first, because it is easier. We can say that B_0 is accountable for all of the error. This is to say that updating the weight will use just the error as the gradient. We can calculate the update for the B_0 coefficient as follows:

$$B_0(t+1) = B_0(t) - \alpha * \text{error}$$

Where $B_0(t+1)$ is the updated version of the coefficient we will use on the next training instance, $B_0(t)$ is the current value for B_0 α is our learning rate and error is the error we calculate for the training instance. Let's use a small learning rate of 0.01 and plug the values into the equation to work out what the new and slightly optimized value of B_0 will be:

$$B_0(t+1) = 0.0 - 0.01 * -1.0$$

$$B_0(t+1) = 0.01$$

Now, let's look at updating the value for B_1 . We use the same equation with one small change. The error is filtered by the input that caused it. We can update B_1 using the equation:

$$B_1(t+1) = B_1(t) - \alpha * \text{error} * x$$

Where $B1(t+1)$ is the update coefficient, $B1(t)$ is the current version of the coefficient, α is the same learning rate described above, $error$ is the same error calculated above and x is the input value.

We can plug in our numbers into the equation and calculate the updated value for $B1$:

$$B1(t+1) = 0.0 - 0.01 * -1 * 1$$

$$B1(t+1) = 0.01$$

We have just finished the first iteration of gradient descent and we have updated our weights to be $B0=0.01$ and $B1=0.01$. This process must be repeated for the remaining 4 instances from our dataset. One pass through the training dataset is called an epoch.

list of all of the values for the coefficients over the 20 iterations

1	B0	B1
2	0.01	0.01
3	0.0397	0.0694
4	0.066527	0.176708
5	0.08056049	0.21880847
6	0.1188144616	0.410078328
7	0.1235255337	0.4147894001
8	0.1439944904	0.4557273134
9	0.1543254529	0.4970511637
10	0.1578706635	0.5076867953
11	0.1809076171	0.6228715633
12	0.1828698253	0.6248337715
13	0.1985444516	0.6561830242
14	0.2003116861	0.6632519622
15	0.1984110104	0.657549935
16	0.2135494035	0.7332419008
17	0.2140814905	0.7337739877
18	0.2272651958	0.7601413984
19	0.2245868879	0.7494281668
20	0.219858174	0.7352420252
21	0.230897491	0.7904386102

A plot of the error for each set of coefficients as the learning process unfolded. This is a useful graph as it shows us that error was decreasing with each iteration and starting to bounce around a bit towards the end.



Final coefficients have the values

$B_0=0.230897491$ and $B_1=0.7904386102$

plug them into our simple linear Regression model and make a prediction for each point in our training dataset.

1	X	y	prediction
2	1	1	0.9551001992
3	2	3	1.690342224
4	4	3	3.160826275
5	3	2	2.42558425
6	5	5	3.8960683

plot our dataset again with these predictions overlaid (x vs y and x vs prediction). Drawing a line through the 5 predictions gives us an idea of how well the model fits the training data.

